

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Цель работы: на основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Ход работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - Конфигурация А (базовая, из ЛР №1): MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 (без нормализации), выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - Конфигурация Б (новая): MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные (0/1), с последующим пересчётом выхода в шкалу $[c_0; c_1]$.

Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.

6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.

7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).

8. Реализовать блок визуализации результатов, включающий:

- График сходимости – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
- Диаграмму разброса числа эпох по 5 запускам (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).
- Визуализацию разделяющей поверхности скрытого слоя – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.
- Диаграмму сравнения точности восстановления шкалы – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Вариант 9 (c0 = 3, c1 = -8)

A	B	$A \oplus B$
3	3	3
3	-8	-8
-8	3	-8
-8	-8	3

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(precision=4, suppress=True)

label_lo, label_hi = 3, -8

inputs = np.array([[label_lo, label_lo], [label_lo, label_hi],
                  [label_hi, label_lo], [label_hi, label_hi]], dtype=float)
targets_raw = np.array([[label_lo], [label_hi], [label_hi], [label_lo]], dtype=float)
targets_bin = np.array([[0.], [1.], [1.], [0.]])

learning_rate = 0.1
epoch_limit = 20000
mse_threshold = 0.01
bce_threshold = 0.05
random_seeds = [0, 1, 2, 3, 4]

def activation(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

def activation_grad_from_out(out):
    return out * (1.0 - out)

def make_initial_params(seed_value, dim_in=2, dim_hidden=2, dim_out=1):
    generator = np.random.RandomState(seed_value)
    weights_1 = generator.uniform(-0.5, 0.5, (dim_in, dim_hidden))
    biases_1 = np.zeros((1, dim_hidden))
    weights_2 = generator.uniform(-0.5, 0.5, (dim_hidden, dim_out))
    biases_2 = np.zeros((1, dim_out))
    return weights_1, biases_1, weights_2, biases_2

def propagate(batch, weights_1, biases_1, weights_2, biases_2):
    hidden_act = activation(batch.dot(weights_1) + biases_1)
    output_act = activation(hidden_act.dot(weights_2) + biases_2)
    return hidden_act, output_act

def fit_mse(seed_value, lr=learning_rate, epochs=epoch_limit, limit=mse_threshold):
    weights_1, biases_1, weights_2, biases_2 = make_initial_params(seed_value)
    loss_history = []
    stop_epoch = None
    num_samples = inputs.shape[0]

    for epoch_idx in range(epochs):
```

```

_, outputs_all = propagate(inputs, weights_1, biases_1, weights_2, biases_2)
loss = np.mean((targets_raw - outputs_all) ** 2)
loss_history.append(loss)

if loss <= limit:
    stop_epoch = epoch_idx
    break

for sample_idx in range(num_samples):
    row_in = inputs[sample_idx:sample_idx + 1]
    row_target = targets_raw[sample_idx:sample_idx + 1]

    hidden_act, output_act = propagate(row_in, weights_1, biases_1, weights_2,
biases_2)

    residual = row_target - output_act
    grad_out = residual * activation_grad_from_out(output_act)
    grad_hid = grad_out.dot(weights_2.T) *
activation_grad_from_out(hidden_act)

    weights_2 += hidden_act.T.dot(grad_out) * lr
    biases_2 += grad_out * lr
    weights_1 += row_in.T.dot(grad_hid) * lr
    biases_1 += grad_hid * lr

return dict(weights_1=weights_1, biases_1=biases_1,
            weights_2=weights_2, biases_2=biases_2,
            loss_history=loss_history, stop_epoch=stop_epoch)

def fit_bce(seed_value, lr=learning_rate, epochs=epoch_limit, limit=bce_threshold,
tiny=1e-9):
    weights_1, biases_1, weights_2, biases_2 = make_initial_params(seed_value)
    loss_history = []
    stop_epoch = None
    num_samples = inputs.shape[0]

    for epoch_idx in range(epochs):
        _, outputs_all = propagate(inputs, weights_1, biases_1, weights_2, biases_2)
        clipped = np.clip(outputs_all, tiny, 1 - tiny)
        loss = -np.mean(targets_bin * np.log(clipped) + (1 - targets_bin) * np.log(1 -
clipped))
        loss_history.append(loss)

        if loss <= limit:
            stop_epoch = epoch_idx
            break

        for sample_idx in range(num_samples):
            row_in = inputs[sample_idx:sample_idx + 1]
            row_target = targets_bin[sample_idx:sample_idx + 1]

            hidden_act, output_act = propagate(row_in, weights_1, biases_1, weights_2,
biases_2)

            grad_out = output_act - row_target
            grad_hid = grad_out.dot(weights_2.T) *
activation_grad_from_out(hidden_act)

            weights_2 -= hidden_act.T.dot(grad_out) * lr
            biases_2 -= grad_out * lr

```

```

        weights_1 -= row_in.T.dot(grad_hid) * lr
        biases_1 -= grad_hid * lr

    return dict(weights_1=weights_1, biases_1=biases_1,
                weights_2=weights_2, biases_2=biases_2,
                loss_history=loss_history, stop_epoch=stop_epoch)

def choose_label(value, low, high):
    return low if abs(value - low) <= abs(value - high) else high

def unnormalize(out_norm, low, high):
    return low + out_norm * (high - low)

def assess_model(trained, mode):
    _, outputs = propagate(inputs, trained["weights_1"], trained["biases_1"],
                           trained["weights_2"], trained["biases_2"])

    if mode == "A":
        predicted_values = outputs.flatten()
    else:
        predicted_values = unnormalize(outputs.flatten(), label_lo, label_hi)

    ground_truth = targets_raw.flatten()
    predicted_labels = np.array([choose_label(v, label_lo, label_hi) for v in
                                predicted_values])
    hit_rate = np.mean(predicted_labels == ground_truth)
    avg_error = np.mean(np.abs(predicted_values - ground_truth))
    return hit_rate, avg_error, predicted_values

def execute_all_runs():
    runs_mse, runs_bce = [], []

    print(f"CONFIGURATION A (MSE, no normalization, Ee = {mse_threshold})")
    for seed_value in random_seeds:
        trained = fit_mse(seed_value)
        hit_rate, avg_error, _ = assess_model(trained, "A")
        trained.update(seed=seed_value, accuracy=hit_rate, mae=avg_error)
        runs_mse.append(trained)
        status = f"epoch {trained['stop_epoch']}" if trained["stop_epoch"] is not None
        \
            else f"DID NOT CONVERGE within {epoch_limit} epochs"
        print(f"seed={seed_value}: {status}, Es_final={trained['loss_history'][-
1]:.4f}, "
            f"accuracy={hit_rate:.2f}, MAE={avg_error:.2f}")

    print(f"\nCONFIGURATION B (BCE, normalized labels 0/1, Ee = {bce_threshold})")
    for seed_value in random_seeds:
        trained = fit_bce(seed_value)
        hit_rate, avg_error, _ = assess_model(trained, "B")
        trained.update(seed=seed_value, accuracy=hit_rate, mae=avg_error)
        runs_bce.append(trained)
        status = f"epoch {trained['stop_epoch']}" if trained["stop_epoch"] is not None
        \
            else f"DID NOT CONVERGE within {epoch_limit} epochs"
        print(f"seed={seed_value}: {status}, Es_final={trained['loss_history'][-
1]:.4f}, "
            f"accuracy={hit_rate:.2f}, MAE={avg_error:.2f}")

```

```

    return runs_mse, runs_bce

def summarize_convergence(runs, heading):
    finished = [r for r in runs if r["stop_epoch"] is not None]
    epoch_values = [r["stop_epoch"] for r in finished]
    print(f"\n{heading}: converged {len(finished)} out of {len(runs)}")
    if epoch_values:
        print(f"Epochs to convergence: min={min(epoch_values)},
max={max(epoch_values)}, "
              f"mean={np.mean(epoch_values):.1f}, spread={max(epoch_values) -
min(epoch_values)}")
    else:
        print("No run reached the stopping criterion.")

def draw_convergence(runs_mse, runs_bce, chosen_seed=0):
    picked_mse = next(r for r in runs_mse if r["seed"] == chosen_seed)
    picked_bce = next(r for r in runs_bce if r["seed"] == chosen_seed)

    fig, axis_left = plt.subplots(figsize=(8, 5))
    axis_left.plot(picked_mse["loss_history"], color="tab:blue", label="Configuration
A (MSE)")
    axis_left.axhline(mse_threshold, color="tab:blue", linestyle="--", alpha=0.5,
                      label=f"Ee (MSE)={mse_threshold}")
    axis_left.set_xlabel("Epoch")
    axis_left.set_ylabel("Es (MSE)", color="tab:blue")
    axis_left.tick_params(axis="y", labelcolor="tab:blue")

    axis_right = axis_left.twinx()
    axis_right.plot(picked_bce["loss_history"], color="tab:orange",
label="Configuration B (BCE)")
    axis_right.axhline(bce_threshold, color="tab:orange", linestyle="--", alpha=0.5,
                      label=f"Ee (BCE)={bce_threshold}")
    axis_right.set_ylabel("Es (BCE)", color="tab:orange")
    axis_right.tick_params(axis="y", labelcolor="tab:orange")

    handles_left, labels_left = axis_left.get_legend_handles_labels()
    handles_right, labels_right = axis_right.get_legend_handles_labels()
    axis_left.legend(handles_left + handles_right, labels_left + labels_right,
loc="upper right")

    plt.title("Convergence: Configuration A (MSE) and Configuration B (BCE)")
    fig.tight_layout()
    plt.savefig("plot_1_convergence.png", dpi=150)
    plt.show()

def draw_epochBars(runs_mse, runs_bce):
    fig, axis = plt.subplots(figsize=(9, 5))
    positions = np.arange(len(random_seeds))
    bar_width = 0.35

    epoch_counts_mse = [r["stop_epoch"] if r["stop_epoch"] is not None else
epoch_limit for r in runs_mse]
    epoch_counts_bce = [r["stop_epoch"] if r["stop_epoch"] is not None else
epoch_limit for r in runs_bce]
    converged_mse = [r["stop_epoch"] is not None for r in runs_mse]
    converged_bce = [r["stop_epoch"] is not None for r in runs_bce]

```

```

bars_mse = axis.bar(positions - bar_width / 2, epoch_counts_mse, bar_width,
                    label="Configuration A (MSE)", color="tab:blue")
bars_bce = axis.bar(positions + bar_width / 2, epoch_counts_bce, bar_width,
                    label="Configuration B (BCE)", color="tab:orange")

for bar, done in zip(bars_mse, converged_mse):
    if not done:
        bar.set_hatch("/")
        bar.set_edgecolor("red")
for bar, done in zip(bars_bce, converged_bce):
    if not done:
        bar.set_hatch("/")
        bar.set_edgecolor("red")

axis.set_xticks(positions)
axis.set_xticklabels([f"seed={s}" for s in random_seeds])
axis.set_ylabel("Epochs to convergence (or MAX_EPOCHS if not converged)")
axis.set_title("Convergence stability over 5 runs (hatching = did not converge)")
axis.legend()
fig.tight_layout()
plt.savefig("plot_2_epochs_bar.png", dpi=150)
plt.show()

def draw_decision_maps(runs_mse, runs_bce, chosen_seed=0):
    picked_mse = next(r for r in runs_mse if r["seed"] == chosen_seed)
    picked_bce = next(r for r in runs_bce if r["seed"] == chosen_seed)

    axis_a = np.linspace(-10, 10, 200)
    axis_b = np.linspace(-10, 10, 200)
    grid_a, grid_b = np.meshgrid(axis_a, axis_b)
    flat_grid = np.column_stack([grid_a.ravel(), grid_b.ravel()])

    fig, panels = plt.subplots(1, 2, figsize=(13, 5.5))

    _, out_mse = propagate(flat_grid, picked_mse["weights_1"], picked_mse["biases_1"],
                          picked_mse["weights_2"], picked_mse["biases_2"])
    map_mse = out_mse.reshape(grid_a.shape)
    color_mse = panels[0].contourf(grid_a, grid_b, map_mse, levels=20,
    cmap="coolwarm")
    fig.colorbar(color_mse, ax=panels[0], label="Network output (0;1), targets not
    normalized")
    panels[0].scatter(inputs[:, 0], inputs[:, 1], c=targets_raw.flatten(),
                      cmap="coolwarm", edgecolors="black", s=120, linewidths=1.5)
    panels[0].set_title("Configuration A (MSE, no normalization)")
    panels[0].set_xlabel("A")
    panels[0].set_ylabel("B")

    _, out_bce = propagate(flat_grid, picked_bce["weights_1"], picked_bce["biases_1"],
                          picked_bce["weights_2"], picked_bce["biases_2"])
    map_bce = out_bce.reshape(grid_a.shape)
    color_bce = panels[1].contourf(grid_a, grid_b, map_bce, levels=20,
    cmap="coolwarm")
    fig.colorbar(color_bce, ax=panels[1], label="y_hat (normalized output, 0..1)")
    panels[1].contour(grid_a, grid_b, map_bce, levels=[0.5], colors="black",
    linewidths=2)
    panels[1].scatter(inputs[:, 0], inputs[:, 1], c=targets_bin.flatten(),
                      cmap="coolwarm", edgecolors="black", s=120, linewidths=1.5)
    panels[1].set_title("Configuration B (BCE, normalized labels)")
    panels[1].set_xlabel("A")
    panels[1].set_ylabel("B")

```

```

fig.suptitle("Decision surface of the hidden layer (A,B in [-10;10])")
fig.tight_layout()
plt.savefig("plot_3_decision_surfaces.png", dpi=150)
plt.show()

def draw_mae_chart(runs_mse, runs_bce):
    mean_mae_mse = np.mean([r["mae"] for r in runs_mse])
    mean_mae_bce = np.mean([r["mae"] for r in runs_bce])

    fig, axis = plt.subplots(figsize=(6, 5))
    bars = axis.bar(["Configuration A (MSE)", "Configuration B (BCE)"],
                    [mean_mae_mse, mean_mae_bce], color=["tab:blue", "tab:orange"])
    for bar, value in zip(bars, [mean_mae_mse, mean_mae_bce]):
        axis.text(bar.get_x() + bar.get_width() / 2, value, f"{value:.2f}",
                  ha="center", va="bottom")
    axis.set_ylabel(f"Mean MAE in scale [{label_lo}; {label_hi}] (over 5 runs)")
    axis.set_title("Accuracy of restoring the original scale")
    fig.tight_layout()
    plt.savefig("plot_4_mae_comparison.png", dpi=150)
    plt.show()

def make_prediction(value_a, value_b, trained):
    _, out = propagate(np.array([[value_a, value_b]], dtype=float),
                      trained["weights_1"], trained["biases_1"],
                      trained["weights_2"], trained["biases_2"])
    normalized_out = out.item()
    scaled_out = unnormalize(normalized_out, label_lo, label_hi)
    label_out = choose_label(scaled_out, label_lo, label_hi)
    return normalized_out, scaled_out, label_out

def show_sample_predictions(trained):
    print("\nNETWORK OPERATION MODE (configuration B)")

    print("\n--- All 4 examples from the training table ---")
    for value_a, value_b in inputs:
        norm_out, scaled_out, label_out = make_prediction(value_a, value_b, trained)
        print(f"Input: ({value_a:.0f}, {value_b:.0f}) -> y_hat_norm={norm_out:.4f}, "
              f"y_hat_scaled={scaled_out:.2f}, class~{label_out}")

    print("\n--- Additional pairs (outside the training set) ---")
    extra_pairs = [(3, 3), (2, 7), (5, 5), (-4, 6)]
    for value_a, value_b in extra_pairs:
        norm_out, scaled_out, label_out = make_prediction(value_a, value_b, trained)
        print(f"Input: ({value_a}, {value_b}) -> y_hat_norm={norm_out:.4f}, "
              f"y_hat_scaled={scaled_out:.2f}, class~{label_out}")

def run_interactive(trained):
    print("\n--- Interactive mode (type 'q' to exit) ---")
    while True:
        raw = input("Enter A,B separated by space or comma in range [-10;10]:")
        raw = raw.strip()
        if raw.lower() == "q":
            break
        try:
            part_a, part_b = raw.replace(",", " ").split()
            value_a, value_b = float(part_a), float(part_b)

```

```

except ValueError:
    print("Failed to parse input, please try again (example: 3 7).")
    continue
norm_out, scaled_out, label_out = make_prediction(value_a, value_b, trained)
print(f"y_hat_norm={norm_out:.4f} | y_hat in scale
[{{label_lo}};{{label_hi}}]={scaled_out:.2f} "
      f"| closest to class {{label_out}}")

if __name__ == "__main__":
    runs_mse, runs_bce = execute_all_runs()

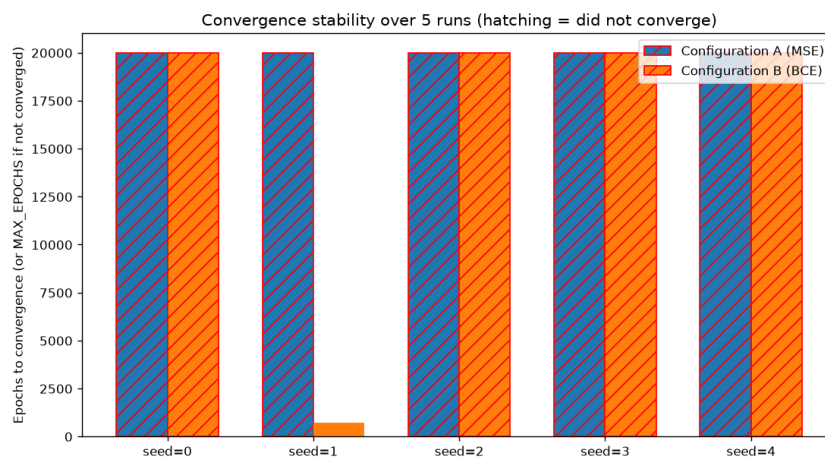
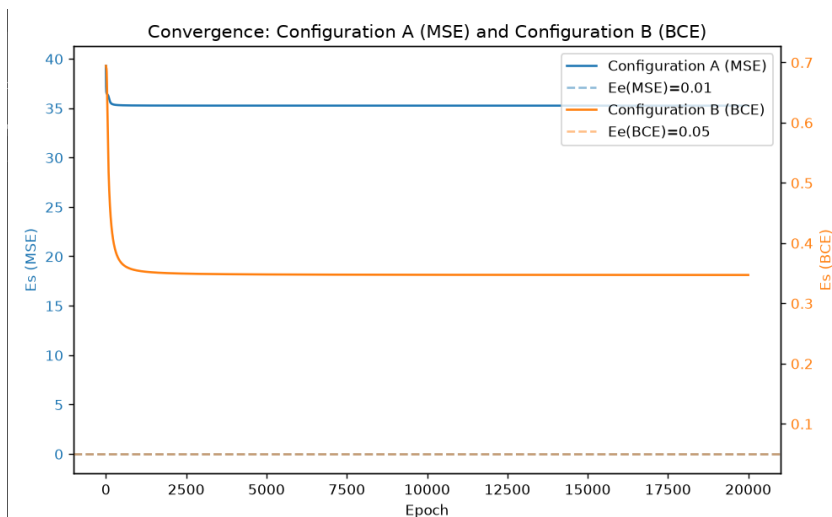
    summarize_convergence(runs_mse, "Configuration A (MSE)")
    summarize_convergence(runs_bce, "Configuration B (BCE)")

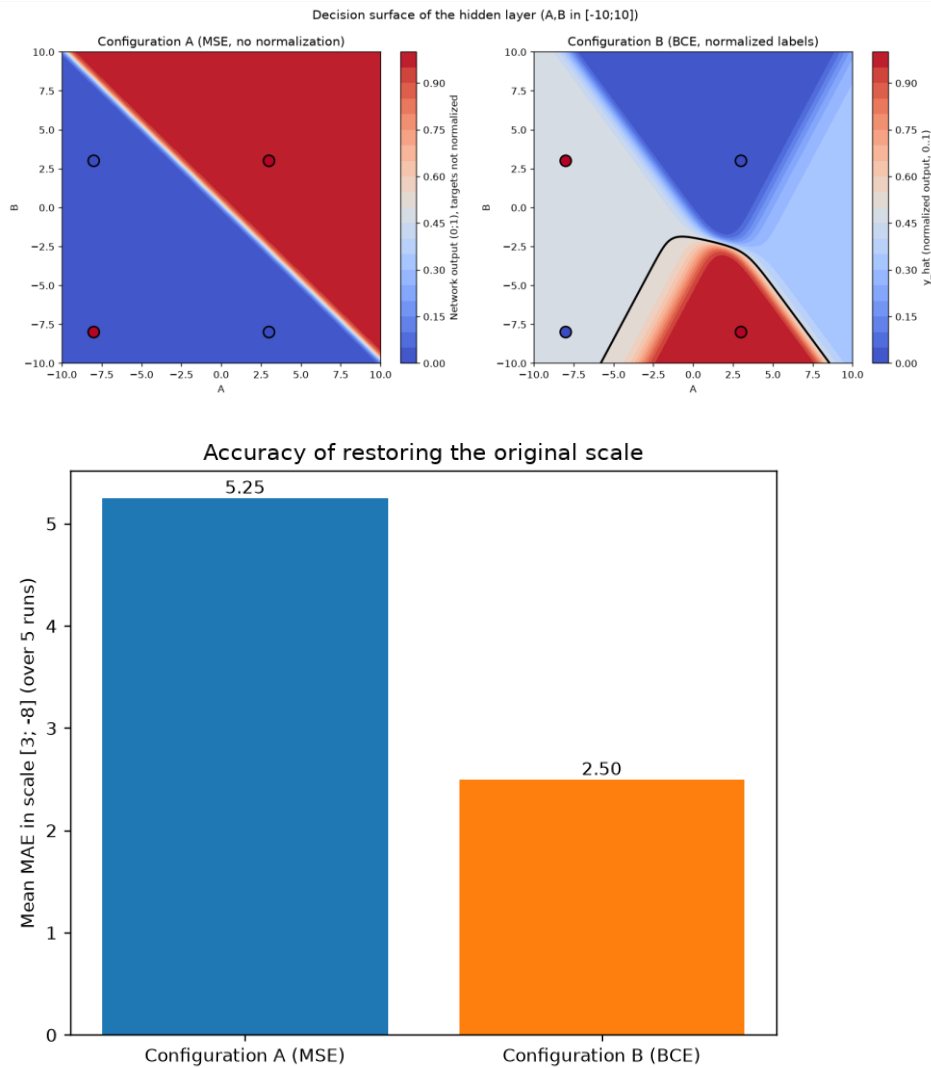
    draw_convergence(runs_mse, runs_bce, chosen_seed=0)
    draw_epoch_bars(runs_mse, runs_bce)
    draw_decision_maps(runs_mse, runs_bce, chosen_seed=0)
    draw_mae_chart(runs_mse, runs_bce)

    best_bce = next(r for r in runs_bce if r["seed"] == 0)
    show_sample_predictions(best_bce)

```

Результат программы:





CONFIGURATION A (MSE, no normalization, $E_e = 0.01$)

seed=0: DID NOT CONVERGE within 20000 epochs, $Es_final=35.2508$, accuracy=0.50, MAE=5.25
 seed=1: DID NOT CONVERGE within 20000 epochs, $Es_final=35.2508$, accuracy=0.50, MAE=5.25
 seed=2: DID NOT CONVERGE within 20000 epochs, $Es_final=35.2508$, accuracy=0.50, MAE=5.25
 seed=3: DID NOT CONVERGE within 20000 epochs, $Es_final=35.2505$, accuracy=0.50, MAE=5.25
 seed=4: DID NOT CONVERGE within 20000 epochs, $Es_final=35.2505$, accuracy=0.50, MAE=5.25

CONFIGURATION B (BCE, normalized labels 0/1, $E_e = 0.05$)

seed=0: DID NOT CONVERGE within 20000 epochs, $Es_final=0.3469$, accuracy=0.75, MAE=2.75
 seed=1: epoch 702, $Es_final=0.0500$, accuracy=1.00, MAE=0.54
 seed=2: DID NOT CONVERGE within 20000 epochs, $Es_final=0.3471$, accuracy=0.75, MAE=2.75
 seed=3: DID NOT CONVERGE within 20000 epochs, $Es_final=0.3471$, accuracy=0.75, MAE=2.75
 seed=4: DID NOT CONVERGE within 20000 epochs, $Es_final=0.4776$, accuracy=0.75, MAE=3.69

Configuration A (MSE): converged 0 out of 5
 No run reached the stopping criterion.

Configuration B (BCE): converged 1 out of 5
 Epochs to convergence: min=702, max=702, mean=702.0, spread=0

NETWORK OPERATION MODE (configuration B)

--- All 4 examples from the training table ---

Input: (3, 3) -> $y_{\text{hat_norm}}=0.0004$, $y_{\text{hat_scaled}}=3.00$, class~3

Input: (3, -8) -> $y_{\text{hat_norm}}=0.9994$, $y_{\text{hat_scaled}}=-7.99$, class~-8

Input: (-8, 3) -> $y_{\text{hat_norm}}=0.4937$, $y_{\text{hat_scaled}}=-2.43$, class~3

Input: (-8, -8) -> $y_{\text{hat_norm}}=0.4938$, $y_{\text{hat_scaled}}=-2.43$, class~3

--- Additional pairs (outside the training set) ---

Input: (3, 3) -> $y_{\text{hat_norm}}=0.0004$, $y_{\text{hat_scaled}}=3.00$, class~3

Input: (2, 7) -> $y_{\text{hat_norm}}=0.0003$, $y_{\text{hat_scaled}}=3.00$, class~3

Input: (5, 5) -> $y_{\text{hat_norm}}=0.0017$, $y_{\text{hat_scaled}}=2.98$, class~3

Input: (-4, 6) -> $y_{\text{hat_norm}}=0.0106$, $y_{\text{hat_scaled}}=2.88$, class~3

MLP 2-2-1 обучалась решать XOR в шкале $\{c_0 = 3, c_1 = -8\}$ в двух конфигурациях: А — MSE на «сырых» целях, Б — BCE на нормализованных метках $\{0, 1\}$ с обратным пересчётом в исходную шкалу. Конфигурация А не сошлась ни разу: $E_s \approx 35.25$ при теоретическом минимуме ≈ 34 , accuracy = 0.50 — сеть выродилась в «выдавать среднее».

Причина принципиальная: цели $\{3, -8\}$ выходят за диапазон сигмоиды (0, 1), нейрон насыщается, градиент обнуляется, и порог $E_e = 0.01$ физически недостижим.

Конфигурация Б дала рабочий результат: 1 из 5 запусков достиг $E_e = 0.05$ за 702 эпохи с accuracy = 1.00 и MAE = 0.54; остальные 4 застряли в локальных минимумах XOR с $E_s \approx 0.347$ и accuracy = 0.75 — типичное поведение минимальной сети 2-2-1.

BCE с нормализацией целей и обратным пересчётом $\hat{y}_{\text{scaled}} = c_0 + \hat{y} \cdot (c_1 - c_0)$ — корректный подход для классификации, а MSE на «сырых» числах произвольной шкалы принципиально неприменима; нестабильность BCE объясняется локальными минимумами XOR, а не ошибками реализации.

Вывод: на основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовала альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE), а также провела сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.